

Sensores e localização em Android

Os sensores são componentes de hardware e software que permitem monitorizar o comportamento do dispositivo móvel ou o ambiente que o rodeia. A maioria dos dispositivos móveis inclui sensores integrados que possibilitam recolher informação sobre o movimento e a posição do dispositivo, bem como sobre as condições ambientais em que este se encontra. A plataforma Android é compatível com sensores de movimento, posição e ambiente.

Os [sensores de movimento](#) permitem monitorizar o movimento do dispositivo. Estes sensores podem também detetar se o dispositivo está inclinado, se foi abanado, entre outras ações. O Android disponibiliza uma vasta gama de sensores, muitos dos quais são sensores de software derivados de sensores de hardware, como o acelerómetro e o giroscópio.

Os [sensores de posição](#) permitem determinar a orientação física do dispositivo no espaço. Para tal, recorrem principalmente ao magnetómetro, que mede o campo magnético terrestre, podendo também combinar informações de outros sensores, como o acelerómetro.

Os [sensores de ambiente](#) permitem monitorizar variáveis do meio onde o dispositivo se encontra, nomeadamente a humidade, a luminosidade, a pressão atmosférica e a temperatura ambiente.

Sensores de movimento	Sensores de posição	Sensores de ambiente
Base hardware: • Acelerómetro (deteta mudanças de velocidade e inclinação) • Giroscópio (mede velocidade de rotação) Base software e hardware: • Gravidade • Aceleração linear • Vetor de rotação • Movimento significativo • Contador de passos • Detetor de passos	Baseados em hardware: • Magnetómetro (mede o campo magnético terrestre)	Base hardware: • Temperatura * • Luminosidade • Pressão atmosférica • Temperatura *Este sensor é suportado pelo Android, mas pouco comum em dispositivos móveis.

Tutorial

Este tutorial tem como objetivo criar uma aplicação Android onde o utilizador pode visualizar um mapa e acompanhar a sua localização em tempo real. Adicionalmente, a aplicação disponibiliza uma funcionalidade de deteção de movimento do utilizador, permitindo identificar se o dispositivo está em movimento ou parado através da análise da magnitude da aceleração.

O objetivo prático deste tutorial é consolidar os seguintes conceitos:

- Integração do Google Maps SDK em aplicações Android;
- Gestão de permissões em tempo de execução, no contexto de localização e visualização da posição do utilizador;
- Integração de sensores em aplicações Android.

A primeira fase do tutorial consiste em integrar o Google Maps na aplicação Android, utilizando o Maps SDK for Android com Jetpack Compose.

- 1) Comece por criar uma *API Key* para o *Maps SDK for Android* através da Google Cloud Console. Siga os passos descritos na documentação oficial: <https://developers.google.com/maps/documentation/android-sdk/get-api-key?setupProd=prerequisites>.
- 2) Dentro do ficheiro *build.gradle (Module)*, adicione a dependência do *Google Maps Compose*, que permite utilizar o Google Maps diretamente em Jetpack Compose. Verifique as versões compatíveis com a sua versão do Kotlin em: <https://github.com/googlemaps/android-maps-compose/releases>

```
dependencies{  
  
    //Google Maps  
    implementation("com.google.maps.android:maps-compose:2.11.4")  
    implementation("com.google.android.gms:play-services-maps:19.1.0")  
  
    //Outras dependências  
}
```

- 3) Para que a aplicação tenha acesso à API criada no passo anterior, é necessário declarar a *API KEY* no ficheiro *AndroidManifest.xml*. Adicione a seguinte meta-data dentro da tag *<application>*, indicando qual a *API Key* que a aplicação deve utilizar para carregar o mapa.

```
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="Adicione_API_KEY " />
```

- 4) Por fim, crie um novo ficheiro chamado *MapScreen.kt*, onde o mapa será apresentado. Neste ficheiro, declare o componente [GoogleMap](#) para mostrar o mapa na aplicação. Nesta primeira fase, o objetivo é apenas garantir que o mapa é carregado corretamente no ecrã e explorar outras funcionalidades do mapa, como a utilização de marcadores.

```
fun MapScreen() {

    val defaultLocation = LatLng(41.6932, -8.8329)

    val cameraPositionState = rememberCameraPositionState {
        position = CameraPosition.fromLatLngZoom(defaultLocation, 12f)
    }

    GoogleMap(
        modifier = Modifier.fillMaxSize(),
        cameraPositionState = cameraPositionState
    ) {
        Marker(
            state = MarkerState(
                position = LatLng(it.latitude, it.longitude)
            ),
            title = "A minha localização"
        )
    }
}
```

A segunda fase consiste em configurar a aplicação para obter a localização do dispositivo e mostrar a posição do utilizador no mapa. Para mostrar a localização do utilizador no mapa, vamos utilizar a funcionalidade nativa do Google Maps, que gere automaticamente a obtenção da localização do dispositivo. Internamente, o Google Maps utiliza o [Fused Location Provider](#).

- 1) Comece por importar a dependência do *Google Play Services Location*. No ficheiro *build.gradle (Module)*, adicione a seguinte dependência.

```
dependencies {  
    implementation("com.google.android.gms:play-services-location:21.3.0")  
}
```

- 2) Para que a aplicação possa aceder à localização do dispositivo, é necessário declarar essa permissão. Comece por declarar a permissão [ACCESS_FINE_LOCATION](#) no *Android Manifest* de forma a obtermos a localização precisa do dispositivo. Repare que ao adicionar esta permissão o IDE apresenta um erro, referindo que ao utilizar a permissão FINE LOCATION deve também adicionar a COARSE LOCATION, que é menos precisa. Adicione as seguintes permissões antes da tag *<application>*.

```
<uses-permission  
    android:name="android.permission.ACCESS_FINE_LOCATION" />  
  
<uses-permission  
    android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

- 3) Sempre que uma aplicação acede à localização do dispositivo, é obrigatório solicitar autorização ao utilizador. De acordo com as boas práticas do Android, a permissão deve ser solicitada apenas quando a [funcionalidade é necessária](#) e não necessariamente quando a aplicação é iniciada. No contexto deste tutorial a localização é apenas utilizada no mapa, por isso este método será lançado apenas no *MapScreen.kt*.

- 4) Dentro do ficheiro *MapScreen.kt*, crie um estado para verificar se a permissão de localização já foi concedida. De acordo com a documentação oficial de [permissões em tempo de execução](#) do Android Developers, utilize o método ***ContextCompat.checkSelfPermission*** para verificar se a permissão foi concedida pelo utilizador.

```
val hasLocationPermission =  
    ContextCompat.checkSelfPermission(  
        context,  
        Manifest.permission.ACCESS_FINE_LOCATION  
    ) == PackageManager.PERMISSION_GRANTED
```

- 5) De seguida, crie o componente responsável por mostrar o pop-up de pedido de permissão ao utilizador. Para isso, utilizamos a [Activity Result API](#), para gerir permissões em tempo de execução. Em Jetpack Compose, esta API pode ser utilizada através do `rememberLauncherForActivityResult`.

```
val permissionLauncher =  
    rememberLauncherForActivityResult(  
        ActivityResultContracts.RequestPermission()  
    ) {}
```

- 6) De seguida crie um [LaunchedEffect](#) para lançar o pedido de permissão quando o MapScreen é criado. Este corre apenas uma vez e gere se o Pop-up é lançado para o utilizador.

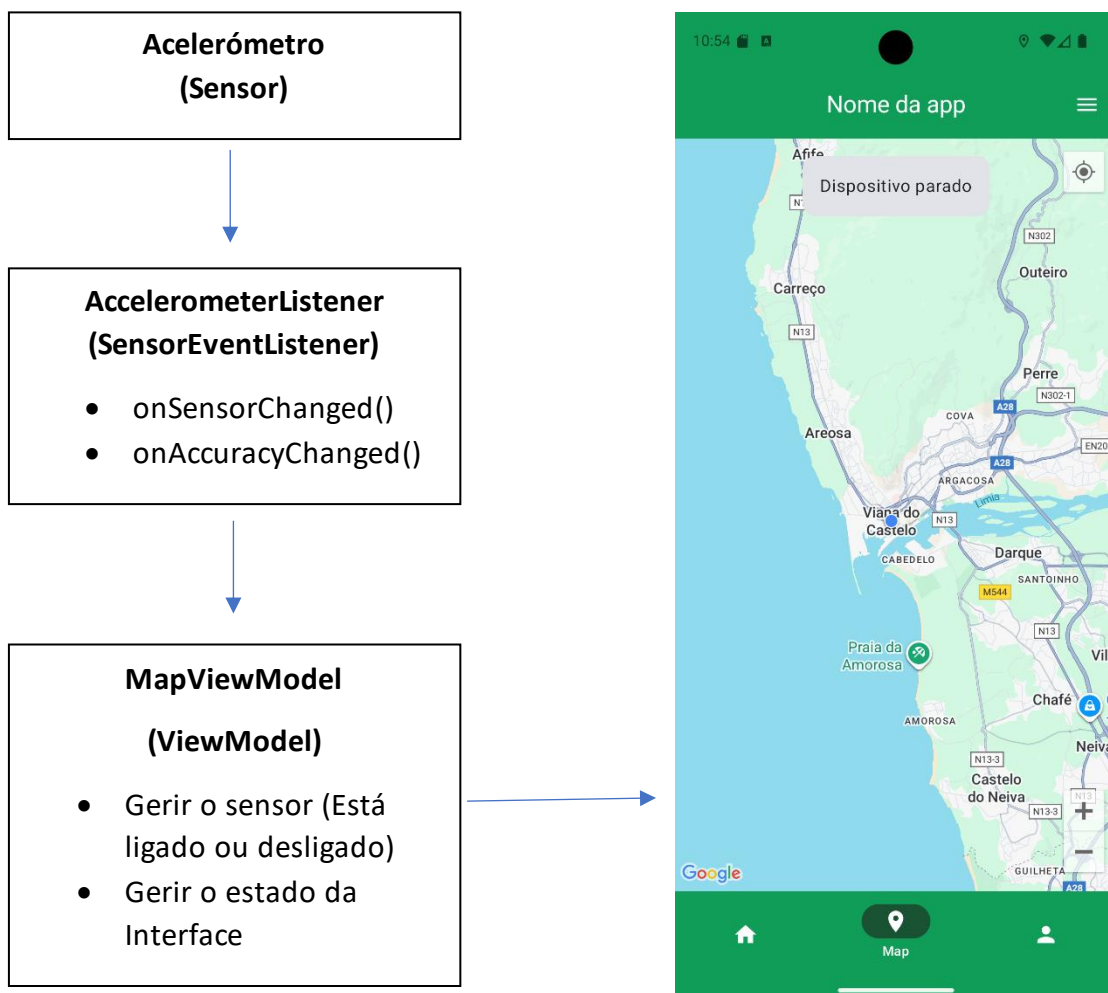
```
LaunchedEffect(Unit) {
    if (!hasLocationPermission) {
        permissionLauncher.launch(
            Manifest.permission.ACCESS_FINE_LOCATION
        )
    }
}
```

- 7) Por fim, configure o componente GoogleMap para mostrar a localização do utilizador. Quando a permissão é concedida, a [My Location Layer](#) é ativada.

```
GoogleMap(
    modifier = Modifier.fillMaxSize(),
    cameraPositionState = cameraPositionState,
    properties = MapProperties(
        isMyLocationEnabled = hasLocationPermission
    )
    //Código do marcador
)
```

A terceira fase consiste em desenvolver uma funcionalidade para detetar o nível de movimento do utilizador e apresentar essa informação sobre o mapa. Para isso, vamos recorrer ao sensor de movimento [TYPE_ACCELEROMETER](#). O acelerómetro é um sensor que mede a aceleração do dispositivo nos três eixos: X – direita/esquerda; Y – cima/baixo; Z – frente/trás. A partir destes valores, é possível calcular a intensidade do movimento do dispositivo.

A utilização de sensores em Android requer a implementação da interface *SensorEventListener*, que permite detetar alterações nos valores do sensor.



- 1) Comece por criar uma classe que implemente a interface [SensorEventListener](#), num novo ficheiro *AccelerometerListener.kt*. A interface *SensorEventListener* exige a implementação dos métodos **onSensorChanged** e **onAccuracyChanged**. Sempre que são detetadas alterações no movimento do dispositivo, a magnitude do movimento é recalculada e essa informação é comunicada ao ViewModel.

```
class AccelerometerListener(  
    private val onMotionChanged: (Boolean) -> Unit  
) : SensorEventListener {  
  
    override fun onSensorChanged(event: SensorEvent) {  
        val x = event.values[0]  
        val y = event.values[1]  
        val z = event.values[2]  
  
        val magnitude = kotlin.math.sqrt(x * x + y * y + z * z)  
        val isMoving = magnitude > 12f  
  
        onMotionChanged(isMoving) //Comunica com o ViewModel  
    }  
  
    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}  
}
```

- 2) De seguida, crie um ViewModel num novo ficheiro chamado *MapViewModel.kt*. Para efeitos deste tutorial, o ViewModel é responsável por: gerir o sensor, calcular o nível de movimento do dispositivo e expor estas alterações de estado à interface gráfica. A interface gráfica utilizará esta informação para mostrar se o dispositivo se encontra em **“Dispositivo em movimento”** ou **“Dispositivo parado”**.

- 3) Começando pelo processo de inicialização do sensor e do estado da interface, adicione o seguinte código ao ViewModel.

```
class MapViewModel(  
    application: Application  
) : AndroidViewModel(application) {  
  
    //Acesso ao sistema de sensores  
    private val sensorManager =  
        application.getSystemService(Context.SENSOR_SERVICE) as  
        SensorManager  
  
    //Especifica o sensor a ser utilizado  
    private val accelerometer =  
        sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)  
  
    //Estado observado pela interface gráfica  
    private val _isMoving = MutableStateFlow(false)  
    val isMoving: StateFlow<Boolean> = _isMoving  
  
    //Listner responsável por receber os dados do sensor. AccelerometerListener  
    //faz parte do SensorEventListener  
    private val listener = AccelerometerListener { moving ->  
        _isMoving.value = moving  
    }  
  
    //Código anterior  
  
}
```

- 4) Ainda no ViewModel, definimos as funções responsáveis por iniciar e parar a recolha de informação do sensor. Este controlo é essencial para evitar o consumo desnecessário de recursos.

```
class MapViewModel(
    application: Application
) : AndroidViewModel(application) {

    //Código anterior

    fun startListening() {
        accelerometer?.let {
            sensorManager.registerListener(
                listener,
                it,
                SensorManager.SENSOR_DELAY_NORMAL
            )
        }
    }

    fun stopListening() {
        sensorManager.unregisterListener(listener)
    }
}
```

- 5) O objetivo é apresentar sobre o mapa a informação que indica se o dispositivo está em movimento ou parado. Dentro do ficheiro MapScreen.kt, definimos a lógica necessária para: ativar o sensor quando este ecrã está em primeiro plano ou desativar o sensor quando o ecrã deixa de estar visível.

Este controlo é feito com base no [ciclo de vida](#) do ecrã. O sensor é ativado no evento ON_START e desativado no evento ON_STOP, seguindo as boas práticas recomendadas pelo Android.

```
@Composable
fun MapScreen(mapViewModel: MapViewModel = viewModel()) {

    //Código anterior

    val isMoving by mapViewModel.isMoving.collectAsState()
    val lifecycleOwner = LocalLifecycleOwner.current

    DisposableEffect(lifecycleOwner) {
        val observer = LifecycleEventObserver { _, event ->
            when (event) {
                Lifecycle.Event.ON_START ->
                    mapViewModel.startListening()

                Lifecycle.Event.ON_STOP ->
                    mapViewModel.stopListening()

                else -> {}
            }
        }

        lifecycleOwner.lifecycle.addObserver(observer)

        onDispose {
            lifecycleOwner.lifecycle.removeObserver(observer)
        }
    }
}
```

- 6) Por fim, para mostrar a informação do sensor no ecrã adicione um cartão sobre o mapa. Este cartão deve utilizar o estado fornecido pelo ViewModel.

```
Box(modifier = Modifier.fillMaxSize()) {
    GoogleMap(
        modifier = Modifier.fillMaxSize()
    )

    Card(
        modifier = Modifier
            .align(Alignment.TopCenter)
            .padding(16.dp)
    ) {
        Text(
            text = if (isMoving)
                "Dispositivo em movimento"
            else
                "Dispositivo parado",
            modifier = Modifier.padding(16.dp)
        )
    }
}
```